P05 Ladders and Hazards

CRE132 worksheet

What you'll make

A short level with **ladders** to climb, **spikes** that hurt, and a **pit** that costs you health and puts you back at the start. When your health runs out, the level restarts.

You'll write two small scripts: one to climb ladders by switching gravity off, and one for the pit. The spikes need no new code at all: they reuse S01's **DamageZone**. All of it is built on **triggers** and **tags**, which is how most "areas" in games work: ladders, water, wind, checkpoints.

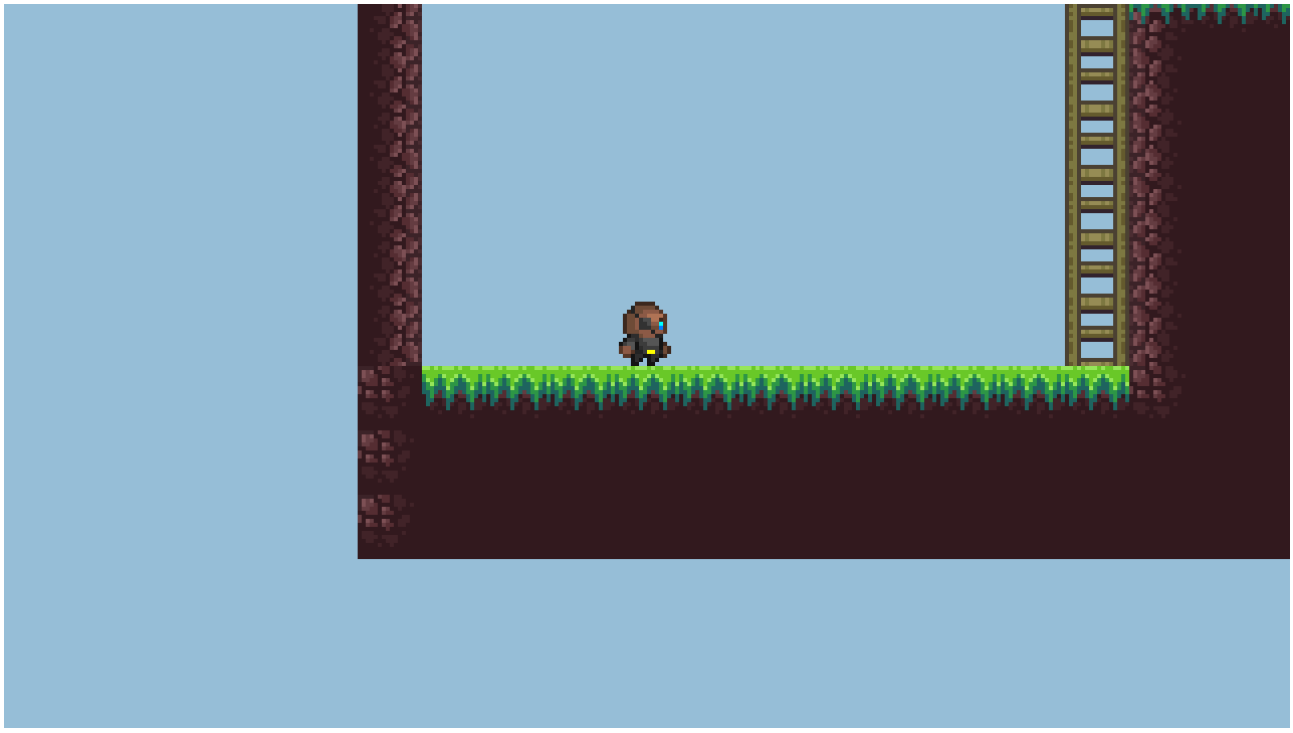


Figure 1: The start of the finished level: the player on the ground and a ladder up to the next ledge.

Before you start

1. Unzip `CRE132_P05_LaddersAndHazards_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P05_LaddersAndHazards_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P05_LaddersAndHazards`.
3. Press **Play**. You can walk (A/D or arrows) and jump (Space) with P01's controller. The spikes already hurt, the health bar goes down, and at zero health the level restarts. But you walk straight past the ladders, and if you fall into the pit you just keep falling.

The player has S01's **Health** (5), **HealthTint** and a health bar from S09. You'll add the ladder climbing and the pit.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. How the ladders are made

Select **Ladders** (a child of **Grid**) and look at the Inspector:

- It's a separate tilemap with a **Tilemap Collider 2D** that has **Is Trigger** ticked. The player can walk through it, but scripts still hear about it.
- Its **Tag** (top of the Inspector) is **Ladder**. The tag is how our script will know this trigger is a ladder and not something else.
- The ladder tiles use the **Grid** collider type, so each cell is a full square and there are no gaps between rungs.

2. Climbing: LadderClimb

Make a folder `Assets/_game/Scripts/P05_LaddersAndHazards` and create a script in it called `LadderClimb`:

```
// Climb ladders with up and down. Ladders are trigger colliders tagged "Ladder".
// While climbing, gravity is switched off and up/down sets the vertical speed.
using UnityEngine;
using UnityEngine.InputSystem;

public class LadderClimb : MonoBehaviour
{
    [SerializeField] private float climbSpeed = 4f;
    [SerializeField] private InputActionReference moveAction;

    private Rigidbody2D body;
    private float normalGravityScale;
    private float verticalInput;
    private bool isTouchingLadder;
    private bool isClimbing;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        normalGravityScale = body.gravityScale;
    }

    private void OnEnable()
    {
        moveAction.action.Enable();
    }

    private void OnDisable()
    {
        moveAction.action.Disable();
    }

    private void Update()
    {
        verticalInput = moveAction.action.ReadValue<Vector2>().y;

        // Start climbing when pressing up or down on a ladder; stop when leaving it.
        if (isTouchingLadder && Mathf.Abs(verticalInput) > 0.5f)
        {
            isClimbing = true;
        }
        if (!isTouchingLadder)
        {
            isClimbing = false;
```

```
        }
    }

    private void FixedUpdate()
    {
        if (isClimbing)
        {
            body.gravityScale = 0f;
            body.linearVelocity = new Vector2(body.linearVelocity.x,
                verticalInput * climbSpeed);
        }
        else
        {
            body.gravityScale = normalGravityScale;
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Ladder"))
        {
            isTouchingLadder = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Ladder"))
        {
            isTouchingLadder = false;
        }
    }
}
```

What's going on:

- **`OnTriggerEnter2D` and `OnTriggerExit2D`** keep `isTouchingLadder` up to date. `CompareTag("Ladder")` ignores every other trigger, such as the spikes.
- **`Mathf.Abs(verticalInput) > 0.5f`**: `Mathf.Abs` removes the minus sign, so pressing up (+1) **or** down (-1) starts climbing. Just standing at a ladder doesn't.
- **While climbing, gravity is 0** and the vertical speed comes straight from the keys. Let go and you stop where you are. Turning gravity off is much simpler than fighting it.
- **`normalGravityScale`** remembers the player's gravity (3) in `Awake`, so it can be put back the moment you leave the ladder.
- **`Update` reads the keys, `FixedUpdate` moves the body**, the same split as the P01 controller.

Add it to the **Player**. Leave **Climb Speed** at **4**. In the **Move Action** slot, click the small circle and pick **Player/Move** (from `CRE132Controls`).

Test it: walk to a ladder and press **W** or **Up**. Climb to the top and step off sideways onto the ledge. Climb part way and let go: you stay put. Press **S** or **Down** to climb down.

3. Spikes: no new code

Select **Spikes** (another tilemap under **Grid**). It has a **Tilemap Collider 2D** with **Is Trigger** ticked and S01's **DamageZone** (**Damage** 1, **Time Between Hits** 1). It's the S01 fire with different art.

- The spike tiles use the **Sprite** collider shape, so only the pointed part hurts, not the empty space around it.

- The **Player's Rigidbody 2D** has **Sleeping Mode** set to **Never Sleep**, so standing still on the spikes keeps hurting (you saw why in S01).

Test it: stand on the spikes and watch the health bar drop once a second. Jump over them instead.

4. The pit: KillZone

Select **Kill Zone**: a long **Box Collider 2D** under the level, with **Is Trigger** ticked. Also find **Respawn Point**, an empty object at the start of the level.

Create `KillZone` in the `P05_LaddersAndHazards` folder:

```
// A trigger below the level (a pit, water, lava). Falling in costs health and puts the
// player back at the respawn point, so one fall isn't the end of the game.
using UnityEngine;

public class KillZone : MonoBehaviour
{
    [SerializeField] private int damage = 1;
    [SerializeField] private Transform respawnPoint;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (!other.CompareTag("Player"))
        {
            return;
        }

        // Move first, then hurt: if this was the last bit of health, the player is
        // destroyed anyway.
        // A Rigidbody2D is moved through its own position, and its fall speed is reset.
        Rigidbody2D body = other.GetComponent<Rigidbody2D>();
        body.position = respawnPoint.position;
        body.linearVelocity = Vector2.zero;

        Health health = other.GetComponent<Health>();
        if (health != null)
        {
            health.TakeDamage(damage);
        }
    }
}
```

- **`if (!other.CompareTag("Player")) return;`** leaves straight away for anything that isn't the player. The `!` means "not".
- **`body.position`** moves the player through its Rigidbody2D, which is the safe way to teleport a physics object.
- **`linearVelocity = Vector2.zero`** cancels the fall. Without it, you'd arrive at the start still falling fast.
- **Move first, then hurt:** `TakeDamage` might be the last hit, and then the player is destroyed. Moving it first means nothing is left half done.

Add it to **Kill Zone**. Leave **Damage** at **1**, and drag **Respawn Point** from the Hierarchy into the **Respawn Point** slot.

Test it: jump into the pit. You appear back at the start, standing still, with one less health.

5. Dying restarts the level

This is already wired up from S09, but have a look. Select the **Player** and find the **Health** component. Its **On Died** event calls **SceneButtons.RestartScene** on the **Level Restarter** object. Health doesn't know about scenes; it just announces that it died, and the Inspector decides what happens.

Test it: fall into the pit five times, or mix pits and spikes. At zero health the level starts again.

Check your work

- ☐ Up or down at a ladder climbs it; letting go stops you; walking off the ladder brings gravity back.
- ☐ Spikes cost 1 health a second, even when standing still.
- ☐ Falling into the pit costs 1 health and puts you back at the start, not falling.
- ☐ At zero health the level restarts.
- ☐ Compare with `CRE132_P05_LaddersAndHazards_Finished` if something behaves differently.

Key points

- **Triggers plus tags make areas.** The ladder doesn't do anything itself: the player's script checks the tag and changes how it moves.
- **Turning gravity off** (`gravityScale = 0`) is often simpler than fighting it. Remember the old value so you can put it back.
- **Reuse:** the spikes are S01's DamageZone with different art. A good component works in many places.

If something goes wrong

- **NullReferenceException in LadderClimb:** the **Move Action** slot is empty.
- **NullReferenceException in KillZone:** the **Respawn Point** slot is empty.
- **Pressing up at the ladder does nothing:** the **Ladders** tilemap's tag isn't **Ladder** (spelling and capital L matter), or its Tilemap Collider 2D isn't a trigger.
- **The pit does nothing:** the **Kill Zone** collider needs **Is Trigger** ticked, and the Player must have the **Player** tag.

Try it

1. Jump off a ladder: when Jump is pressed while climbing, stop climbing. (Hint: you'll need a `jumpAction` too, and to set `isClimbing` to false.)
2. Make a checkpoint: a trigger further along that moves the **Respawn Point** to itself when the player touches it. (P10 does this properly.)
3. Make the spikes knock the player back as well as hurting. (P09 does this.)